oldver\dataset3.py

```python
# ──────────────────────────────────────────────────────────────────────────
# Dataset + augmentation
#   - geometric : RandomResizedCrop, flip, small rotation/translation
#   - colour    : ColorJitter + per-channel RGB gain the P_hat perturbation
#   - noise     : Gaussian noise on the normalised tensor
# ──────────────────────────────────────────────────────────────────────────
import json
import torch
import numpy as np
from PIL import Image
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
from torchvision import transforms

from config3 import (
    IMAGE_SIZE, NUM_WORKERS, STATS_PATH, NUM_CLASSES,
    COLOR_JITTER, CHANNEL_SCALE, NOISE_STD,
)

# ── Load leak-safe normalisation stats  ──────
try:
    with open(STATS_PATH, "r", encoding="utf-8") as f:
        _s = json.load(f)
    MEAN, STD = _s["mean"], _s["std"]
except Exception:
    MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]


# ── Custom transforms class-based => picklable for Windows workers ──────────
class RandomChannelScale:
    """Multiply each RGB channel by an independent random gain in [1-a, 1+a].
    This reproduces the professor's P_hat colour perturbation: it makes an image
    look redder / greener / bluer, exactly the distortion expected at test time."""
    def __init__(self, amount: float = 0.15):
        self.a = amount
    def __call__(self, img: Image.Image) -> Image.Image:
        arr = np.asarray(img, dtype=np.float32)
        gain = np.random.uniform(1 - self.a, 1 + self.a, size=3).astype(np.float32)
        arr = np.clip(arr * gain, 0, 255).astype(np.uint8)
        return Image.fromarray(arr)


class AddGaussianNoise:
    """Add zero-mean Gaussian noise to a normalised tensor (TRAIN ONLY)."""
    def __init__(self, std: float = 0.04):
        self.std = std
    def __call__(self, t: torch.Tensor) -> torch.Tensor:
        return t + torch.randn_like(t) * self.std

class ConvertRGB:
    """Convert PIL image to RGB (handles RGBA). Picklable for Windows workers."""
    def __call__(self, img: Image.Image) -> Image.Image:
        return img.convert("RGB")

def build_train_transform():
    b, c, s, h = COLOR_JITTER
    return transforms.Compose([
        ConvertRGB(),
        transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.7, 1.0)),  # patch+scale
        transforms.RandomHorizontalFlip(p=0.5),                   # reflection
        transforms.RandomRotation(degrees=15),                    # rotation
        transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)), # translation
        transforms.ColorJitter(brightness=b, contrast=c, saturation=s, hue=h),
        RandomChannelScale(CHANNEL_SCALE),                        # P_hat colour shift
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
        AddGaussianNoise(NOISE_STD),                              # noise (train only)
        transforms.RandomErasing(p=0.25),                         # occlusion / cutout
    ])


```

```python
def build_val_transform():
    return transforms.Compose([
        ConvertRGB(),                                          # RGBA -> RGB
        transforms.Resize(int(IMAGE_SIZE * 1.14)),
        transforms.CenterCrop(IMAGE_SIZE),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])


train_transform = build_train_transform()
val_transform   = build_val_transform()


class VirusDataset(Dataset):
    """Reads (filepath, label) rows from the split dataframe."""
    def __init__(self, df, transform=None):
        self.df = df.reset_index(drop=True)
        self.transform = transform
    def __len__(self):
        return len(self.df)
    def __getitem__(self, i):
        row = self.df.iloc[i]
        img = Image.open(row["filepath"]).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, int(row["label"])


def make_weighted_sampler(train_df) -> WeightedRandomSampler:
    """Oversample minority class (class 3) so every batch is class-balanced."""
    counts = np.bincount(train_df["label"].values, minlength=NUM_CLASSES)
    class_w = 1.0 / (counts + 1e-8)
    sample_w = class_w[train_df["label"].values]
    return WeightedRandomSampler(
        weights=torch.as_tensor(sample_w, dtype=torch.double),
        num_samples=len(sample_w), replacement=True,
    )


def get_dataloaders(train_df, val_df, batch_size):
    train_ds = VirusDataset(train_df, transform=train_transform)
    val_ds   = VirusDataset(val_df,   transform=val_transform)
    sampler  = make_weighted_sampler(train_df)
    train_loader = DataLoader(
        train_ds, batch_size=batch_size, sampler=sampler,   # sampler => no shuffle
        num_workers=NUM_WORKERS, pin_memory=True, drop_last=False,
    )
    val_loader = DataLoader(
        val_ds, batch_size=batch_size, shuffle=False,
        num_workers=NUM_WORKERS, pin_memory=True,
    )
    return train_loader, val_loader
```